Informe técnico — Segunda entrega del TPI Food Store

Grupo: Victoria Guzman, Daniela Sosa, Octavio Skumanic, Sebastián Vivanco, Guillermo Sanchez

Base de Datos II · Unidades 4 a 8 · 10/11/2026 — Motores: PostgreSQL 17.11 (PL/pgSQL), MongoDB 8.0.15, SQLite 3.43

Toda la entrega se instala y se prueba con un solo comando sobre una base nueva: `python tpi2/ejecutar_todo.py`. La corrida registrada (`evidencias/20261110_204915/`) terminó con **132 comprobaciones correctas de 132, en 67 s**.

1. Qué se implementó

Unidad	Elementos
4 · Normalización	Esquema heredado (`legado.venta_planilla`, 819 renglones con anomalías reales); validación de 9 dependencias funcionales contra los datos; descomposición a BCNF con 3 tablas nuevas (`zona`, `repartidor`, `entrega`); migración transaccional con cuarentena, registro de discrepancias y mapas de trazabilidad; verificación de reunión sin pérdida; reversión en tres niveles; inventario de desnormalizaciones con su salvaguarda. Detalle
5 · Seguridad	Tabla `usuario` con bcrypt, `fn_autenticar` con bloqueo y `fn_resetear_password`; 9 roles de grupo y 7 de login con mínimo privilegio, permisos por columna y por defecto; auditoría del servidor y auditoría por fila de solo agregado; copias anonimizadas; simulacro de acceso indebido con log anonimizado, reconstrucción por IA y contención; auditoría de seguridad asistida por IA con 11 controles. Auditoría · Incidente · Permisos · Equivalencia
6 · Mantenimiento	Respaldo diario verificado (rol de solo lectura, SHA-256, roles), restauración en modo prueba y siniestro, prueba semanal de restauración, mantenimiento (`VACUUM ANALYZE`, refrescos, métricas), verificación de integridad física y lógica (amcheck, checksums), programación en el Programador de tareas; simulacro de desastre. Plan · Post-incidente
7 · NoSQL	Colección `pedidos` con 2.398 documentos reales del proyecto, validador `$jsonSchema` + `$expr`, `Decimal128`, 3 índices; 8 consultas en cada motor comparadas automáticamente; medición de la duplicación. Comparación
8 · Móvil	Base SQLite `STRICT` de la tablet; cliente que descarga el catálogo, arma borradores, encola y sincroniza; función de servidor idempotente que decide y descuenta stock; 12 casos probados, incluidos conflictos y concurrencia entre dos tablets. Detalle
Cierre	Specs de las cinco unidades en formato Kiro (`.kiro/specs/`); code review de los scripts de toda la cursada (28 hallazgos con decisión); bitácora de IA y DUIA

2. Cómo se probó

- **Transacciones reversibles.** Cada script de cambio se ejecuta sin `BEGIN`/`COMMIT` propios, dentro de una transacción del ejecutor. La migración de la U4 se ensayó con `ROLLBACK`, se confirmó, se revirtió con `05_reversion.sql`, se probó un error provocado a mitad de camino y se volvió a aplicar. En cada paso se comparó una **huella de contenido** (filas y md5 de cada tabla): el `ROLLBACK`, el error provocado y la reversión dejaron la base idéntica byte a byte.
- **Copias de prueba.** El ejecutor crea siempre una base nueva con nombre fechado y nunca reutiliza ni pisa una existente. La restauración de prueba usa otra base temporal y la elimina.
- **Pruebas positivas y negativas de permisos.** 26 operaciones con 7 roles distintos (lo que cada rol debe poder hacer y lo que no), más la verificación de los 75 privilegios previstos contra el catálogo: 0 faltantes, 0 sobrantes.
- **Simulacro de acceso indebido.** 4 fallos y un acierto contra `fn_autenticar`, 3 fallos y un acierto de conexión directa, lectura masiva, modificación y 7 intentos de escalamiento.
- **Simulacro de desastre.** `DROP DATABASE` de la base en uso, restauración contra reloj, comparación de huella, permisos y prueba de humo de la aplicación.
- **Comparación entre motores.** Resultados completos de 8 consultas comparados valor por valor entre PostgreSQL y MongoDB; 9 pruebas del validador.
- **Casos de conflicto de sincronización.** Precio cambiado, sin stock, baja lógica, respuesta perdida, sin red, dos tablets concurrentes, borrador vacío, instantánea vacía y datos inválidos.

3. Resultados

Prueba	Resultado
Migración U4	819 renglones = 817 migrados + 2 en cuarentena; importe $4.488.032,00 en ambos lados; **0 diferencias** en la reconstrucción; 2 discrepancias registradas; desnormalización U4 sincronizada
Permisos	75 privilegios previstos, 0 faltantes, 0 sobrantes; 26/26 pruebas por rol; objetos futuros sin acceso salvo decisión explícita
Simulacro de acceso	**7/7 escalamientos rechazados.** La alerta de fuerza bruta detectó el ataque; la contención bloqueó el rol, invalidó la clave adivinada y restauró el dato alterado desde la auditoría
Anonimización	29 grupos idénticos entre datos reales y anónimos; seudónimos 1 a 1 (360/360)
Simulacro de desastre	**RTO 7,94 s** (objetivo 15 min): verificación 0,17 s, restauración 4,19 s, estadísticas 1,05 s, integridad 1,04 s. **RPO 11,9 s** (objetivo 24 h). Perdidos: 3 pedidos y 1 cambio de precio posteriores al respaldo. Informe post-incidente
Respaldo y restauración	Respaldo de 220.543 bytes en 3,34 s; respaldo fallido sin archivo residual; respaldo alterado rechazado por SHA-256
MongoDB	2.398/2.398 documentos cargados; **8/8 consultas con resultados idénticos**; 8 documentos inválidos rechazados y `_id` duplicado con E11000
Sincronización	12/12 casos; en la concurrencia, la segunda tablet esperó 1,48 s el bloqueo y fue rechazada; stock final 1; cada solicitud aplicada tiene exactamente un pedido

Hallazgo de la U6 que vale destacar: `log_statement = mod` **no registra** `CALL`. Los pedidos perdidos en el desastre no se pueden reconstruir desde el log, mientras que el cambio de precio sí. Se propone archivar WAL para recuperación a un instante (PITR) y auditar también las altas de pedidos.

4. Decisiones de diseño

- **Refactorización (U4).** Se validaron las dependencias antes de normalizar. Cuatro de las nueve candidatas no se cumplen en los datos y por eso no son claves: el precio es de la venta (queda en la línea), una zona tiene varios repartidores, el teléfono de un cliente cambió (se conserva el último y se registra el anterior) y hubo doble carga (se suman cantidades tras verificar que el precio coincide). Se distinguió "diferencia de formato" de "diferencia de dato": `PIZZAS` y `Pizzas` son lo mismo; `Pizas` también, pero eso lo decidió una persona, no la migración.
- **Desnormalización controlada.** Se mantienen cinco, cada una con su salvaguarda (tabla en normalizacion_bcnf.md). Se decidió **no** guardar el total del pedido en PostgreSQL.
- **Roles y permisos (U5).** Un rol por función y uno de login por persona o sistema. La aplicación registra pedidos solo por procedimiento y no tiene `UPDATE` sobre precios ni stock: las reglas que necesitaban bloquear filas pasaron a `SECURITY DEFINER` con dueño sin privilegios especiales. Nadie tiene `DELETE`: las bajas son lógicas. Soporte ve al personal sin el hash. Reportes solo ve agregados y copias anónimas.
- **Documental vs. relacional (U7).** PostgreSQL sigue siendo la fuente de verdad: las reglas que importan son referenciales (cliente y producto existen, producto activo, stock, transiciones) y MongoDB no puede garantizarlas al escribir. MongoDB queda como vista de lectura derivada, regenerable, solo si aparece una necesidad concreta de lectura masiva del pedido completo.
- **Sincronización (U8).** "La tablet propone, el servidor dispone": la tablet no escribe tablas del servidor. Ante un cambio de precio gana el servidor; ante falta de stock o baja, el rechazo es definitivo. La idempotencia por UUID evita ventas duplicadas al reintentar. La venta de feria descuenta stock con bloqueo para que dos tablets no vendan la misma unidad.

5. Uso de IA

Se usó **Claude Code** (Anthropic) para especificar, programar, ejecutar las pruebas, auditar y documentar. **No se usaron Kiro ni OpenCode**: las specs tienen el formato de Kiro pero se redactaron con Claude Code. Prompts, specs, propuestas aceptadas y descartadas, y los siete errores de la IA detectados por las pruebas están en la bitácora y la DUIA. Entre lo descartado: un `GRANT UPDATE` amplio para la aplicación, dar a reportes una vista con nombres de clientes, devolver `TEMPORARY` a `PUBLIC`, importes `double` en MongoDB y, en la contención, bloquear una IP que era de documentación. El error más significativo fue de la reconstrucción del incidente: vinculó dos identidades que el log no relaciona.